

Домашнє завдання. Модуль 9, Урок 3

Тема: Швидкість API, HTTP-кешування та підготовка до highload

Контекст: У Уроках 9.1 і 9.2 ти оптимізував CRM: TransferRepository, InvoiceRepository, AccountRepository з eager loading і пагінацією, ServiceService з кешем і інвалідацією, Job GenerateTransferSummaryReport, індекси, CACHE_STRATEGY. PM поставив завдання: **підготувати API CRM до highload** - додати HTTP-кешування (Cache-Control) для довідників і списків, rate limiting для захисту від перевантаження, контролювати об'єм відповідей через Resources, винести експорт виписки в Job і звести все в **Production-Ready API Checklist**. Це завершує Модуль 9 і **всю програму**: від архітектури та API (Модулі 5–7) через логування та моніторинг (Модуль 8) до оптимізації та підготовки до зростання навантаження.

Мета

Перевірити й закріпити: швидкість API як продукт, HTTP-кешування (Cache-Control), контроль об'єму відповіді (Resources, пагінація), rate limiting, Jobs для важких операцій, stateless API, Production-Ready checklist.

Передумова

Проект `crm-laravel` з Module 9.1–9.2: TransferRepository, InvoiceRepository, AccountRepository з eager loading і пагінацією, ServiceService з кешем, GenerateTransferSummaryReport Job, CACHE_STRATEGY, PERFORMANCE_ANALYSIS. Resources (TransferResource, InvoiceResource, AccountResource) існують. API-ендпоінти transfers, invoices, accounts, services.

Завдання 1. HTTP-кешування: Cache-Control для ендпоінтів CRM

Опис: Додай Cache-Control до відповідей GET-ендпоінтів CRM згідно з типом даних.

Бізнес-контекст: HTTP-кешування дозволяє клієнту (браузер, мобільний додаток) або проксі зберігати відповідь і не звертатися до сервера при повторних запитах протягом max-age. Для довідника послуг (services) - публічний кеш на годину; для списку transfers - короткий private кеш (15 сек), щоб не показувати застарілі дані довго. Це зменшує навантаження на API та прискорює повторні запити.

Що реалізувати:

1. Доповни `docs/API_OPTIMIZATION.md` (створи документ) розділом «HTTP-кешування»
2. **GET /api/v1/services** (довідник послуг - публічний, рідко змінюється):

```
$response->header('Cache-Control', 'public, max-age=3600');
```

3. **GET /api/v1/transfers** (персоналізований/часто змінюваний список):

```
$response->header('Cache-Control', 'private, max-age=15');
```

4. **GET /api/v1/invoices** - аналогічно transfers: **private, max-age=15**
5. **GET /api/v1/accounts** - **private, max-age=15**
6. **POST, PATCH, DELETE** - не додавати Cache-Control або явно **no-store** для мутацій
7. **GET /api/v1/accounts/{id}** (баланс рахунку) - **Cache-Control: no-store** або **private, max-age=0** - не кешувати критичні фінансові дані
8. Таблиця в **API_OPTIMIZATION.md**:

Ендпоінт	Cache-Control	Причина
GET /api/v1/services	public, max-age=3600	Публічний довідник, рідко змінюється
GET /api/v1/transfers	private, max-age=15	Персоналізований, часто змінюється
GET /api/v1/invoices	private, max-age=15	Аналогічно
GET /api/v1/accounts	private, max-age=15	Аналогічно
GET /api/v1/accounts/{id}	no-store	Баланс - real-time
POST /api/v1/transfers	(не кешується)	Мутація

Критерії прийняття:

- ☐ Є API_OPTIMIZATION.md з розділом «HTTP-кешування»
- ☐ GET /api/v1/services має **public, max-age=3600**
- ☐ GET /api/v1/transfers, invoices, accounts мають **private, max-age=15**
- ☐ GET /api/v1/accounts/{id} має **no-store** або **private, max-age=0**
- ☐ Є таблиця в документі

Завдання 2. Rate limiting для API CRM

Опис: Налаштуй rate limiting для API-ендпоінтів CRM з різними лімітами для звичайних та важких операцій.

Бізнес-контекст: Без rate limiting один клієнт або зловмисник може генерувати тисячі запитів і вивести API з ладу. Ліміти захищають систему та забезпечують справедливий доступ. Для звичайних GET/POST - 60 req/min; для важких операцій (генерація звіту, експорт) - 5 req/min.

Що реалізувати:

1. Онови **routes/api.php** або **bootstrap/app.php** (Laravel 11) - middleware throttle для API
2. Стандартний ліміт для більшості ендпоінтів:

```
Route::middleware(['auth:sanctum', 'throttle:60,1'])->group(function  
( ) {  
    Route::get('transfers', [TransferController::class, 'index']);  
    Route::get('transfers/{id}', [TransferController::class, 'show']);  
    Route::post('transfers', [TransferController::class, 'store']);  
    Route::get('invoices', [InvoiceController::class, 'index']);  
    // ...  
});
```

throttle:60,1 - 60 запитів на хвилину

3. Жорсткіший ліміт для важких операцій:

```
Route::middleware(['auth:sanctum', 'throttle:5,1'])->group(function ()
{
    Route::post('reports/transfer-summary', [ReportsController::class,
'generateTransferSummary']);
    Route::post('reports/export-statement', [ReportsController::class,
'exportStatement']);
});
```

throttle:5,1 - 5 запитів на хвилину

4. **Публічний ендпоінт** (наприклад, GET /api/v1/services без auth) - **throttle:120,1** (або окремий ліміт)

5. **Заголовки відповіді:** Laravel автоматично додає X-RateLimit-Limit, X-RateLimit-Remaining; при 429 - Retry-After

6. **Доповни API_OPTIMIZATION.md** розділом «Rate limiting» з таблицею лімітів

Критерії прийняття:

- ☐ Є throttle для API (60/min для звичайних)
- ☐ Є окремий throttle (5/min) для reports/export
- ☐ Є документація в API_OPTIMIZATION.md
- ☐ При перевищенні повертається 429
- ☐ Є заголовки X-RateLimit-*

Завдання 3. Контроль об'єму відповіді: TransferResource та InvoiceResource

Опис: Переконайся, що TransferResource та InvoiceResource повертають лише потрібні поля для клієнта; при потребі створи окремі «list» версії з обмеженим набором полів.

Бізнес-контекст: Великий JSON сповільнює мережеву передачу та серіалізацію. Зайві поля (внутрішні ідентифікатори, raw-відповіді шлюзів, updated_at для списку) не потрібні клієнту і збільшують розмір. Resources контролюють структуру; окремий TransferListItemResource для списку може містити менше полів, ніж TransferResource для деталей.

Що реалізувати:

1. Проаналізуй поточні TransferResource та InvoiceResource:

- Які поля повертаються?
- Чи є зайві для клієнта (gateway_id, internal_notes, raw_response тощо)?

2. Створи або оновить **TransferListItemResource** (для списку transfers):

- id, account_from_id, account_to_id, amount, currency, status, description, created_at
- Вкладені account_from (id, account_number), account_to (id, account_number) - без client/details

3. Створи або оновить **InvoiceListItemResource** (для списку invoices):

- id, client_id, total_amount, status, created_at
 - Вкладені client (id, name) - без повного профілю
4. **Використовуй** TransferListItemResource/InvoiceListItemResource у index()-методах контролерів замість повного Resource
5. **Доповни API_OPTIMIZATION.md** розділом «Об'єм відповіді» - перелік полів для списку vs деталі

Критерії прийняття:

- ☐ Є TransferListItemResource (або оновлено TransferResource) з обмеженим набором полів
- ☐ Є InvoiceListItemResource (або оновлено InvoiceResource)
- ☐ Немає зайвих полів (internal, gateway_raw тощо)
- ☐ Контролери використовують list Resources для index
- ☐ Документ оновлено

Завдання 4. Job: ExportAccountStatementJob

Опис: Створи Job для експорту виписки по рахунку (transfers за період у CSV). API приймає запит, ставить Job, повертає 202 Accepted з task_id.

Бізнес-контекст: Генерація CSV виписки за місяць при тисячах transfers може займати 10–30 секунд. Синхронний виконання в HTTP-запиті призводить до таймаутів. Job виконується асинхронно; API швидко відповідає; клієнт отримує результат по task_id або нотифікацією.

Що реалізувати:

1. Створи Job **App\Jobs\ExportAccountStatementJob**:

```
<?php

declare(strict_types=1);

namespace App\Jobs;

use App\Models\Account;
use App\Models\Transfer;
use Illuminate\Bus\Queueable;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Foundation\Bus\Dispatchable;
use Illuminate\Queue\InteractsWithQueue;
use Illuminate\Queue\SerializesModels;
use Illuminate\Support\Facades\Storage;

final class ExportAccountStatementJob implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    public function __construct(
        public readonly int $accountId,
        public readonly string $dateFrom,
        public readonly string $dateTo,
```

```

        public readonly int $requestedByUserId,
        public readonly string $taskId
    ) {}

    public function handle(): void
    {
        $transfers = Transfer::query()
            ->with(['accountFrom', 'accountTo'])
            ->where(function ($q) {
                $q->where('account_from_id', $this->accountId)
                    ->orWhere('account_to_id', $this->accountId);
            })
            ->whereBetween('created_at', [$this->dateFrom, $this->dateTo])
            ->orderBy('created_at')
            ->get();

        $path = "exports/statement_{$this->taskId}.csv";
        // Генерація CSV, збереження в Storage
        // Опційно: оновити запис task з path, відправити notification

        $this->saveCsv($path, $transfers);
    }

    private function saveCsv(string $path, $transfers): void
    {
        // Простий CSV: id, date, amount, currency, type,
        account_from, account_to
        $content =
        "id,date,amount,currency,type,account_from,account_to\n";
        foreach ($transfers as $t) {
            $type = $t->account_from_id === $this->accountId ? 'out' :
            'in';

            $content .= implode(',', [
                $t->id,
                $t->created_at->format('Y-m-d H:i'),
                $t->amount ?? '',
                $t->currency ?? '',
                $type,
                $t->accountFrom->account_number ?? '',
                $t->accountTo->account_number ?? '',
            ]) . "\n";
        }
        Storage::put($path, $content);
    }
}

```

2. API-ендпоінт **POST /api/v1/reports/export-statement:**

- Тіло: { "account_id": 1, "date_from": "2026-01-01", "date_to": "2026-01-31" }
- Валідація (FormRequest)
- Генерація taskId (Str::uuid())

- `ExportAccountStatementJob::dispatch(...)`
 - Відповідь 202: `{ "status": "accepted", "task_id": "...", "message": "Export started" }`
3. **Опційно:** `GET /api/v1/reports/export-statement/{task_id}` - перевірка статусу або посилання на файл (якщо збережено в Storage)
 4. **Rate limiting:** `throttle:5,1` для цього ендпоінту (Завдання 2)
 5. **Доповни `API_OPTIMIZATION.md`** розділом «Jobs для важких операцій»

Критерії прийняття:

- ☐ Є `ExportAccountStatementJob`
- ☐ Job генерує CSV та зберігає в Storage
- ☐ Є ендпоінт `POST /api/v1/reports/export-statement`
- ☐ Відповідь 202 з `task_id`
- ☐ Rate limit 5/min для експорту
- ☐ Документ оновлено

Завдання 5. Production-Ready API Checklist

Опис: Збери всі вимоги до production-ready API CRM в один checklist.

Бізнес-контекст: РМ хоче єдиний чеклист: що має бути зроблено, щоб API CRM вважалось готовим до production та highload. Це основа для code review, деплою та онбордингу. Це завершення всієї програми.

Що реалізувати:

1. Створи `docs/PRODUCTION_READY_CHECKLIST.md`

2. Чеклист (з усіх модулів):

| # | Категорія | Вимога | Статус | ||--||--||--|| **API (Модуль 7)** | | 1 | Контракт | REST, versioning `/api/v1` | ☐ | | 2 | Валідація | `FormRequest` для мутацій | ☐ | | 3 | Відповіді | `Resources/DTO`, єдиний формат помилок | ☐ | | 4 | Документація | OpenAPI, контракт задокументований | ☐ | | 5 | Тести | Feature-тести для API | ☐ | | **Модуль 8** | | 6 | Логування | Структуровані логи, Correlation ID | ☐ | | 7 | Error tracking | Sentry, фільтр очікуваних помилок | ☐ | | 8 | Інциденти | INCIDENT_RUNBOOK, post-mortem template | ☐ | | **Модуль 9** | | 9 | БД | Eager loading, пагінація, індекси | ☐ | | 10 | Кеш | Redis, кеш довідників, інвалідація | ☐ | | 11 | HTTP-кеш | Cache-Control для відповідних ендпоінтів | ☐ | | 12 | Rate limiting | Throttle для API, окремий для важких операцій | ☐ | | 13 | Об'єм | `Resources` з обмеженим набором полів | ☐ | | 14 | Jobs | Важкі операції (звіти, експорт) в чергах | ☐ | | **Highload** | | 15 | Stateless | Аутентифікація по токєну, сесія в Redis | ☐ | | 16 | Кеш/черги | Redis для кешу та черг (зовні застосунку) | ☐ |

3. **Правило:** перед деплоєм в production перевірити, що критичні пункти виконані
4. **Посилання** на PERFORMANCE_ANALYSIS, CACHE_STRATEGY, API_OPTIMIZATION, INCIDENT_RUNBOOK

Критерії прийняття:

- ☐ Є `PRODUCTION_READY_CHECKLIST.md`

- ☐ Є мінімум 14 пунктів
- ☐ Є категорії (API, Модуль 8, Модуль 9, Highload)
- ☐ Є правило про перевірку перед деплоєм
- ☐ Є посилання на інші документи

Завдання 6. Stateless API: короткий аудит

Опис: Переконайся, що API CRM є stateless і готовий до горизонтального масштабування.

Бізнес-контекст: При highload додають кілька інстансів застосунку за балансувальником. Щоб це працювало, API має бути stateless: не покладатися на локальний стан (сесія в пам'яті одного процесу). Аутентифікація по токєну (Sanctum), сесія в Redis або БД - дозволяють будь-якому інстансу обробити запит.

Що реалізувати:

1. **Доповни API_OPTIMIZATION.md** розділом «Stateless API»
2. **Чеклист:**
 - ☐ Аутентифікація по Bearer токєну (Sanctum) - не по cookie/session в файлі
 - ☐ Сесія (якщо є) зберігається в Redis або БД, не в file
 - ☐ Кеш у Redis (не array/file для production)
 - ☐ Черги в Redis (не sync для production)
3. **Опиши:** якщо всі пункти виконані, API готовий до горизонтального масштабування - балансувальник може направляти запити на будь-який інстанс
4. **Конфіг:** SESSION_DRIVER=redis, CACHE_DRIVER=redis, QUEUE_CONNECTION=redis для production

Критерії прийняття:

- ☐ Є розділ «Stateless API»
- ☐ Є чеклист (3+ пункти)
- ☐ Є опис готовності до масштабування
- ☐ Згадано конфіг для production

Завдання 7. Підсумковий документ: що має API CRM після Модуля 9

Опис: Створи короткий підсумковий документ про стан API CRM після всіх модулів.

Бізнес-контекст: РМ хоче єдиний огляд: що реалізовано, які документи існують, як це пов'язано з моніторингом та оптимізацією. Це підсумок всієї програми.

Що реалізувати:

1. **Доповни docs/README.md** або створи docs/API_OVERVIEW.md
2. **Структура:**
 - **API:** ендпоінти transfers, invoices, accounts, services, reports
 - **Архітектура:** Controller → Service → Repository, DTO, Resources
 - **Оптимізація:** eager loading, пагінація, кеш довідників, індекси, Jobs
 - **HTTP:** Cache-Control, rate limiting
 - **Моніторинг:** логи, Sentry, INCIDENT_RUNBOOK

- **Документи:** OpenAPI, PERFORMANCE_ANALYSIS, CACHE_STRATEGY, API_OPTIMIZATION, PRODUCTION_READY_CHECKLIST

3. **Зв'язок з модулями:** Модуль 5 (Jobs), 6 (архітектура, тести), 7 (API), 8 (логи, Sentry), 9 (продуктивність)

4. **Метрики для перевірки:** час відповіді (Telescope, логи), кількість запитів до БД, rate limit 429

Критерії прийняття:

- ☐ Є API_OVERVIEW.md (або оновлено README)
- ☐ Є опис API, архітектури, оптимізації, моніторингу
- ☐ Є зв'язок з модулями
- ☐ Є згадка метрик

Формат здачі

Структура проекту (доповнення):

```
crm-laravel/  
├── app/Http/Resources/Api/V1/  
│   ├── TransferListItemResource.php  
│   └── InvoiceListItemResource.php  
├── app/Jobs/  
│   └── ExportAccountStatementJob.php  
├── app/Http/Controllers/Api/V1/  
│   └── ReportsController.php - exportStatement, generateTransferSummary  
├── routes/  
│   └── api.php - throttle middleware  
├── docs/  
│   ├── API_OPTIMIZATION.md - HTTP-кеш, rate limit, об'єм, jobs  
│   ├── PRODUCTION_READY_CHECKLIST.md  
│   └── API_OVERVIEW.md - підсумок  
└── (оновлені контролери з Cache-Control)
```

Перевірка:

- GET /api/v1/services повертає Cache-Control: public, max-age=3600
- GET /api/v1/transfers повертає Cache-Control: private, max-age=15
- Rate limit: 429 при перевищенні
- POST /api/v1/reports/export-statement повертає 202 з task_id
- ExportAccountStatementJob генерує CSV
- PRODUCTION_READY_CHECKLIST існує
- API_OPTIMIZATION задокументований

Рекомендації РМ

- HTTP-кеш доповнює Redis: для публічних довідників - public, max-age=3600; для персональних списків - короткий private.
- Rate limiting - захист для всіх; без нього API вразливий до зловживань та витоку.
- Jobs для експорту - API завжди відповідає швидко; важка робота йде у фоні.

- Production-Ready Checklist - основа для стабільності; використовуй перед кожним релізом.

Це завершення Модуля 9 і всієї програми. Після виконання домашнього завдання ти маєш повний цикл: архітектура, API, тести, логування, моніторинг, оптимізація БД, кеш, HTTP-кеш, rate limiting, Jobs, підготовка до highload.

Орієнтовний час виконання: 150–180 хвилин